

VULNERABILITY ASSESSMENT REPORT

Penetration Testing & Security Research

WordPress User Enumeration via REST API

CVE Class: Information Disclosure / Improper Access Control
CWE-200: Exposure of Sensitive Information to an Unauthorised Actor
CVSS v3.1 Score: 5.3 (Medium)

SEVERITY

MEDIUM

Report Reference: VAR-2025-WP-007

Date of Assessment: 14 July 2025

Classification: CONFIDENTIAL

Target Domain: [REDACTED] — withheld per responsible disclosure policy

1. Executive Summary

The target web application, a WordPress installation, was found to be vulnerable to unauthenticated user enumeration through the WordPress REST API. The `/wp-json/wp/v2/users` endpoint is publicly accessible without any form of authentication, returning a structured JSON response that discloses registered user accounts in full.

The exposed data includes user IDs, display names, URL slugs (which function as login usernames), author archive links, and Gravatar avatar URLs. This vulnerability requires no special tooling, privileges, or conditions to exploit — a simple HTTP GET request is sufficient to retrieve the complete user list.

Security Impact

This finding directly reduces the effort required to conduct a credential-based attack against the application. An attacker in possession of valid usernames can immediately pivot to targeted brute-force, credential stuffing, or spear-phishing operations without requiring any prior access to the system.

Vulnerability Summary

Vulnerability Name	WordPress User Enumeration via REST API
Severity	Medium
CVSS v3.1 Score	5.3 — CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CWE Classification	CWE-200: Exposure of Sensitive Information to an Unauthorised Actor
Vulnerability Class	Information Disclosure / Improper Access Control
Target Application	WordPress CMS — domain redacted per responsible disclosure
Affected Endpoint	<code>/wp-json/wp/v2/users</code>
Authentication Required	None — fully unauthenticated request
Date Identified	14 July 2025
Report Reference	VAR-2025-WP-007
Status	Open — Awaiting Remediation

2. Technical Details

The following section documents the methodology applied during identification of this vulnerability, the commands and tools used, and the server’s observed behaviour.

2.1 Reconnaissance

Initial passive footprinting of the target confirmed the platform as a WordPress installation. Indicators observed included the presence of the `/wp-content/` and `/wp-includes/` directory paths in page source, the `X-Powered-By` response header, and the default WordPress login path at `/wp-login.php`. No active scanning was required at this stage.

2.2 REST API Testing

The WordPress REST API is enabled by default in all installations from version 4.7 onwards. The `/wp/v2/users` namespace, when accessible to unauthenticated requests, returns a paginated list of registered users. The following command was issued from a standard testing workstation:

```
# Unauthenticated GET request to the WordPress REST API users endpoint
curl -s https://[REDACTED].com/wp-json/wp/v2/users | jq

# Server Response: HTTP/1.1 200 OK
# Content-Type: application/json; charset=UTF-8
```

2.3 Observed Server Response

The application returned HTTP 200 OK with a JSON array containing fully populated user objects. No authentication headers, session tokens, or cookies were present in the request. The following fields were disclosed for each registered user:

- `id` — Internal WordPress user ID (integer)
- `name` — User display name / full name
- `slug` — URL slug, which directly corresponds to the WordPress login username
- `link` — Full URL to the user's author archive page
- `avatar_urls` — Gravatar image URLs at multiple resolutions
- `_links` — HAL-style hypermedia links exposing further API endpoints

3. Proof of Concept (PoC)

The following JSON excerpt represents the structure of the response returned by the server. All personally identifiable values have been partially redacted in this report to comply with responsible disclosure obligations. The full, unredacted evidence is available upon request by the affected party.

```
[
  {
    "id": 1,
    "name": "r****", // Display name redacted
```

```
    "slug": "r*****", // Login username redacted
    "link": "https://[REDACTED].com/author/r*****/",
    "avatar_urls": {
      "96": "https://secure.gravatar.com/avatar/..."
    },
    "_links": { ... }
  },
  {
    "id": 2,
    "name": "S***** A**", // Display name redacted
    "slug": "s*****", // Login username redacted
    "link": "https://[REDACTED].com/author/s*****/",
    "avatar_urls": {
      "96": "https://secure.gravatar.com/avatar/..."
    },
    "_links": { ... }
  }
]
```

Note on Redaction

The target domain and all user-identifiable data (full names, usernames, avatar hashes, and profile URLs) have been redacted in this published report. Evidence is retained securely and available to the affected organisation upon verified request.

4. Impact Analysis

While this vulnerability does not directly result in account compromise, it constitutes a critical enabler in a multi-stage attack scenario. The practical consequences are assessed below across three dimensions:

4.1 Increased Credential Attack Efficiency

In a brute-force or credential-stuffing attack, the attacker must supply both a valid username and the correct password. By eliminating the need to guess or enumerate usernames through other means, this vulnerability removes an entire phase of the attack lifecycle. An adversary can immediately commence targeted password attacks using the exposed slugs as login identifiers.

4.2 Targeted Social Engineering and Phishing

The disclosure of full display names, in conjunction with the platform context, enables the construction of highly convincing spear-phishing campaigns. Administrative users identified through this endpoint are of particular interest as targets, given their elevated privileges within the application.

4.3 Organisational Reconnaissance

The enumeration of user accounts reveals the number of registered users, their roles (via author archive pages), and the internal naming conventions of the organisation. This information contributes materially to a broader intelligence-gathering operation and reduces the preparation time for subsequent attacks.

Confidentiality	LOW — Non-sensitive but operationally valuable user metadata exposed
Integrity	NONE — No data is modified by this vulnerability in isolation
Availability	NONE — No service disruption in isolation
Attack Complexity	LOW — Single unauthenticated HTTP GET request required
Privileges Required	NONE — No account or session required

5. Remediation Recommendations

The following controls are recommended in order of implementation priority. Controls at the application layer are preferred over network-level controls, as they address the root cause rather than providing a supplementary barrier.

5.1 Restrict REST API Access to Authenticated Users (Priority: High)

The most effective remediation is to require authentication for all REST API requests. The following code should be added to the active theme's functions.php file or a site-specific plugin:

```
add_filter( 'rest_authentication_errors', function( $result ) {
    if ( ! is_user_logged_in() ) {
        return new WP_Error(
            'rest_not_logged_in',
            'Authentication is required to access the REST API.',
            array( 'status' => 401 )
        );
    }
    return $result;
} );
```

5.2 Specifically Filter the Users Endpoint (Priority: High)

If full REST API restriction is not feasible, the `/wp-json/wp/v2/users` and `/wp-json/wp/v2/users/{id}` endpoints should be explicitly blocked or redirected for unauthenticated requests. This can be achieved via server-level configuration (nginx/Apache rewrite rules) or within WordPress using the `rest_endpoints` filter.

5.3 Deploy a WordPress Security Plugin (Priority: Medium)

Security plugins such as Wordfence Security or Sucuri Security include dedicated settings to prevent user enumeration via both the REST API and the legacy `?author=` query parameter

method. Enabling the 'Prevent User Enumeration' feature within these plugins addresses this vulnerability without requiring custom code.

5.4 Configure Web Application Firewall Rules (Priority: Medium)

A Web Application Firewall (WAF) should be configured to block or challenge unauthenticated external requests that target the WordPress REST API users namespace. Rule criteria should match requests containing `/wp-json/wp/v2/users` in the URL path originating from outside the application's administrative IP ranges.

5.5 Address Legacy Enumeration via Author Parameter

It should be noted that WordPress also permits user enumeration via the `/?author=N` query string (e.g., `/?author=1`), which triggers a redirect to the corresponding author archive URL, leaking the username. This vector should be blocked concurrently with the REST API fix to prevent an attacker from using an alternative enumeration method.

6. References ---

- WordPress Developer Documentation — REST API Users Endpoint: <https://developer.wordpress.org/rest-api/reference/users/>
- OWASP Testing Guide — OTG-IDENT-004: Testing for Account Enumeration
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor — <https://cwe.mitre.org/data/definitions/200.html>
- CVSS v3.1 Specification Document — FIRST.org
- Wordfence Security Plugin — User Enumeration Prevention: <https://www.wordfence.com>

End of Report

VAR-2025-WP-007 — WordPress User Enumeration via REST API — CONFIDENTIAL

This report was prepared for responsible disclosure purposes. The target domain has been redacted.